Study 2 — Wave4+ Clean Pipeline (Attention + Hardcoded Longstring)

0) Config

- **Edit the `path`** below to your CSV export.
- Column names are **hard-coded** to your Wave4+ survey. If your export uses different labels, adjust in the **Column Map** section.
- This file creates:
  - `FirstItem` … `SeventhItem` (1–4: position of selected choice)
  - `longstring_run` via `{careless}` across those seven items
  - `exclude_reason` with `"attention_fail"` and `"longstring_outlier"`
  - `dat_final` after all exclusions

```
# ---- PATH ----
path <- r"(D:\Dropbox\USD\Dissertation\Collab folder\Data\Dissertation+-+Study+2+-+Wave4+_October+6,+20
```

1) Packages

```
if (!requireNamespace("pacman", quietly = TRUE)) install.packages("pacman")
pacman::p_load(readr, dplyr, stringr, ggplot2, broom, effectsize, careless)
```

2) Load

```
raw0 <- readr::read_csv(
  path,
  col_types = cols(.default = col_character()),
  show_col_types = FALSE
)

# Drop the first two header-ish rows (labels + ImportId JSON)
stopifnot(nrow(raw0) >= 3)
dat <- raw0[-c(1,2), , drop = FALSE]

# Keep finished only (exact header "Finished")
dat <- dat %>% filter(Finished %in% c("True","true","1"))
```

```r
# Coerce duration and score
dat[["Duration (in seconds)"]] <- suppressWarnings(as.numeric(dat[["Duration (in seconds)"]]))
dat[["SC0"]]                   <- suppressWarnings(as.numeric(dat[["SC0"]]))
dat$score <- dat[["SC0"]]

stopifnot(nrow(dat) > 0)
stopifnot(!all(is.na(dat$score)))
```

3) Derive factors (Version → hierarchy & chartjunk)

```r
stopifnot("Version" %in% names(dat))
dat$Version <- stringr::str_squish(dat$Version)

chart <- stringr::str_match(dat$Version, "(High|Medium|Low)Chartjunk$")[,2]
hier  <- stringr::str_remove(dat$Version, "(High|Medium|Low)Chartjunk$")

dat$chartjunk <- factor(chart, levels = c("Low","Medium","High"), ordered = TRUE)
dat$hierarchy <- factor(
  dplyr::recode(hier,
    "NoHierarchy"         = "NoHierarchy",
    "FloorplanHierarchy"  = "FloorplanHierarchy",
    "RoomHierarchy"       = "RoomHierarchy",
    "ImportanceHierarchy" = "ImportanceHierarchy",
    .default = NA_character_
  ),
  levels = c("NoHierarchy","FloorplanHierarchy","RoomHierarchy","ImportanceHierarchy")
)
```

4) Column Map (hard-coded)

These are the exact headers expected in your Wave4+ export. Adjust here if your CSV labels differ.

```r
# === Column Map (Wave4+ CSV using QIDs) ===
# AN display order (tokens like Q1|Q9|...|MetaInfo)
COL_AN      <- "Gistblock_DO"

# Seven items (Q1 & Q18 have per-row randomized choice order)
COL_Q1      <- "Q1"
COL_Q1_DO   <- "Q1_DO"

COL_Q9      <- "Q9"
COL_Q10     <- "Q10"
COL_Q14     <- "Q14"
COL_Q17     <- "Q17"

COL_Q18     <- "Q18"
COL_Q18_DO  <- "Q18_DO"
```

```r
# Attention item (also one of the seven)
COL_Q20     <- "Q20"

# Fixed (non-DO) choice orders for 4-option questions
# (coerce source columns to character so match() works even if CSV parsed numerics)
dat[[COL_Q10]] <- as.character(dat[[COL_Q10]])
dat[[COL_Q14]] <- as.character(dat[[COL_Q14]])
dat[[COL_Q17]] <- as.character(dat[[COL_Q17]])
dat[[COL_Q9 ]] <- trimws(as.character(dat[[COL_Q9 ]]))
dat[[COL_Q20]] <- trimws(as.character(dat[[COL_Q20]]))

CHO_Q9  <- c("North","South","East","West")
CHO_Q10 <- c("0","1","2","3")
CHO_Q14 <- c("1","2","3","4")
CHO_Q17 <- c("0","1","2","3")
CHO_Q20 <- c("Smart home","IT infrastructure","Hospital bed display","Stock ticker")

req_cols <- c(COL_AN,COL_Q1,COL_Q1_DO,COL_Q9,COL_Q10,COL_Q14,COL_Q17,COL_Q18,COL_Q18_DO,COL_Q20)
if (!all(req_cols %in% names(dat))) {
  stop("Missing columns: ", paste(setdiff(req_cols, names(dat)), collapse = ", "))
}
```

5) Attention filter (Q20 must contain “smart home”)

```r
keep_idx <- grepl("smart[[:space:]]*home", dat[[COL_Q20]], ignore.case = TRUE)
keep_idx[is.na(keep_idx)] <- FALSE

dat$exclude_reason <- ifelse(keep_idx, NA_character_, "attention_fail")
dat_attn <- dat[keep_idx, , drop = FALSE]

message(sprintf("Attention filter kept %d / %d rows.", sum(keep_idx, na.rm = TRUE), nrow(dat)))
```

```
## Attention filter kept 883 / 1063 rows.
```

6) Build FirstItem…SeventhItem (1–4 positions)

Rules: - Parse `COL_AN` (pipe-separated tokens) → drop `MetaInfo` → take the first seven tokens as the **seen order**. - Map each token to one of `{Q1, Q9, Q10, Q14, Q17, Q18, Q20}`. - For Q1 & Q18, compute the selected **position** from the per-row “- Display Order” columns. - For fixed-order items, compute the position via `match(selected, CHO_*)`.

```r
# 6a) Parse AN into seven tokens per row
ORD_LIST <- strsplit(dat_attn[[COL_AN]], "\\|", perl = TRUE)
ORD_LIST <- lapply(ORD_LIST, function(v) {
  v <- v[!grepl("^\\s*MetaInfo\\s*$", v, ignore.case = TRUE)]
  if (length(v) >= 7) v[1:7] else c(v, rep(NA_character_, 7 - length(v)))
})
```

```r
# 6b) Selected positions per item (1..4)

# Q1 and Q18: position derived from their row-specific Display Order
POS_Q1 <- as.integer(mapply(function(ans, do) {
  if (is.na(ans) || is.na(do)) return(NA_integer_)
  ch <- strsplit(do, "\\|", perl = TRUE)[[1]]
  match(ans, ch)
}, dat_attn[[COL_Q1]], dat_attn[[COL_Q1_DO]]))

POS_Q18 <- as.integer(mapply(function(ans, do) {
  if (is.na(ans) || is.na(do)) return(NA_integer_)
  ch <- strsplit(do, "\\|", perl = TRUE)[[1]]
  match(ans, ch)
}, dat_attn[[COL_Q18]], dat_attn[[COL_Q18_DO]]))

# Fixed-order items
POS_Q9  <- as.integer(match(dat_attn[[COL_Q9]],  CHO_Q9))
POS_Q10 <- as.integer(match(dat_attn[[COL_Q10]], CHO_Q10))
POS_Q14 <- as.integer(match(dat_attn[[COL_Q14]], CHO_Q14))
POS_Q17 <- as.integer(match(dat_attn[[COL_Q17]], CHO_Q17))
POS_Q20 <- as.integer(match(dat_attn[[COL_Q20]], CHO_Q20))

# Mapping token → vector of positions in fixed order c(Q1,Q9,Q10,Q14,Q17,Q18,Q20)
QKEYS <- c("Q1","Q9","Q10","Q14","Q17","Q18","Q20")

# Helper inline (no separate functions): index into the packed vector by token
pick_by_token <- function(tok, p1,p9,p10,p14,p17,p18,p20) {
  idx <- match(tok, QKEYS)
  c(p1,p9,p10,p14,p17,p18,p20)[idx]
}

FirstItem   <- as.integer(mapply(function(ord, p1,p9,p10,p14,p17,p18,p20) pick_by_token(ord[1], p1,p9,p
                                 ORD_LIST, POS_Q1, POS_Q9, POS_Q10, POS_Q14, POS_Q17, POS_Q18, POS_Q20))
SecondItem  <- as.integer(mapply(function(ord, p1,p9,p10,p14,p17,p18,p20) pick_by_token(ord[2], p1,p9,p
                                 ORD_LIST, POS_Q1, POS_Q9, POS_Q10, POS_Q14, POS_Q17, POS_Q18, POS_Q20))
ThirdItem   <- as.integer(mapply(function(ord, p1,p9,p10,p14,p17,p18,p20) pick_by_token(ord[3], p1,p9,p
                                 ORD_LIST, POS_Q1, POS_Q9, POS_Q10, POS_Q14, POS_Q17, POS_Q18, POS_Q20))
FourthItem  <- as.integer(mapply(function(ord, p1,p9,p10,p14,p17,p18,p20) pick_by_token(ord[4], p1,p9,p
                                 ORD_LIST, POS_Q1, POS_Q9, POS_Q10, POS_Q14, POS_Q17, POS_Q18, POS_Q20))
FifthItem   <- as.integer(mapply(function(ord, p1,p9,p10,p14,p17,p18,p20) pick_by_token(ord[5], p1,p9,p
                                 ORD_LIST, POS_Q1, POS_Q9, POS_Q10, POS_Q14, POS_Q17, POS_Q18, POS_Q20))
SixthItem   <- as.integer(mapply(function(ord, p1,p9,p10,p14,p17,p18,p20) pick_by_token(ord[6], p1,p9,p
                                 ORD_LIST, POS_Q1, POS_Q9, POS_Q10, POS_Q14, POS_Q17, POS_Q18, POS_Q20))
SeventhItem <- as.integer(mapply(function(ord, p1,p9,p10,p14,p17,p18,p20) pick_by_token(ord[7], p1,p9,p
                                 ORD_LIST, POS_Q1, POS_Q9, POS_Q10, POS_Q14, POS_Q17, POS_Q18, POS_Q20))

dat_attn$FirstItem   <- FirstItem
dat_attn$SecondItem  <- SecondItem
dat_attn$ThirdItem   <- ThirdItem
dat_attn$FourthItem  <- FourthItem
dat_attn$FifthItem   <- FifthItem
dat_attn$SixthItem   <- SixthItem
dat_attn$SeventhItem <- SeventhItem
```

7) Longstring via {careless}

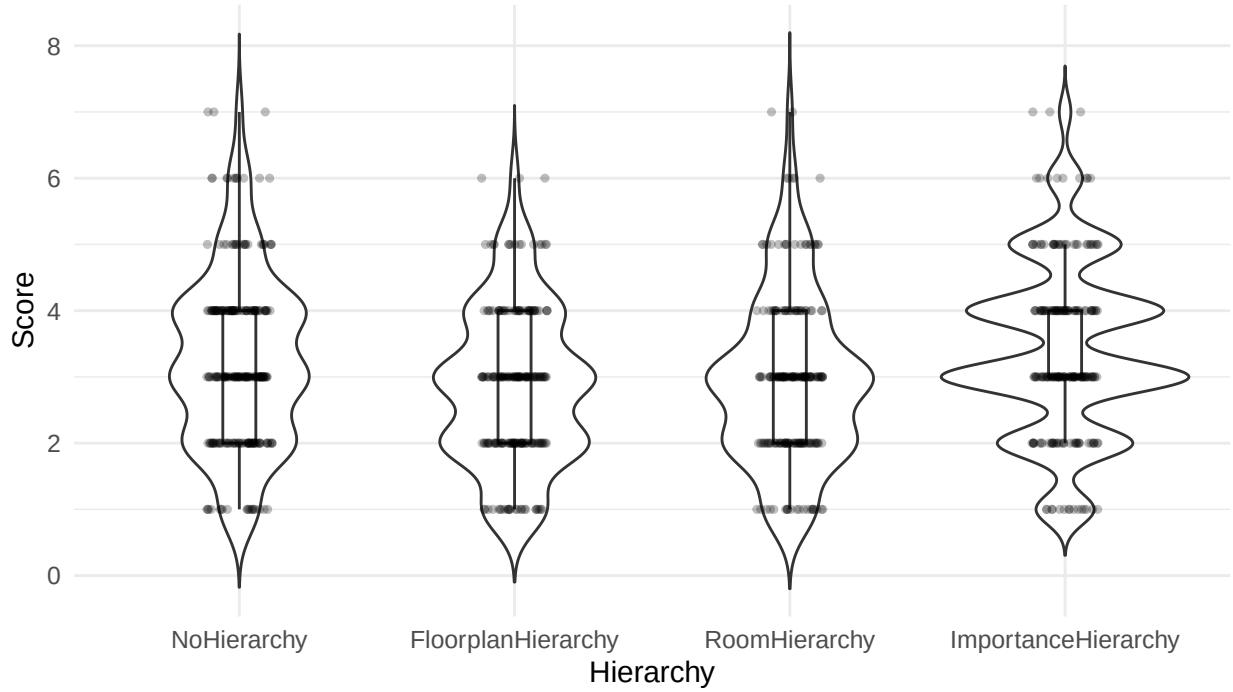
We compute longest run across the seven 1–4 codes. To avoid NA runs, we replace missing entries with **unique per-column sentinels** so consecutive NAs never match.

```
ls_mat <- dat_attn[, c("FirstItem","SecondItem","ThirdItem","FourthItem","FifthItem","SixthItem","Seven

# Clean blanks → NA_integer_
for (j in seq_along(ls_mat)) {
  x <- ls_mat[[j]]
  x[x %in% c("", "NA")] <- NA
  ls_mat[[j]] <- suppressWarnings(as.integer(x))
}

# Replace NA with unique per-column codes (breaks NA runs)
for (j in seq_along(ls_mat)) {
  idx <- is.na(ls_mat[[j]])
  if (any(idx)) ls_mat[[j]][idx] <- -j
}

dat_attn$longstring_run <- as.integer(careless::longstring(ls_mat))

# Tukey fence to flag outliers; set fixed cutoff if preferred
q3  <- stats::quantile(dat_attn$longstring_run, 0.75, na.rm = TRUE)
iqr <- stats::IQR(dat_attn$longstring_run, na.rm = TRUE)
thr <- q3 + 1.5 * iqr
flag <- !is.na(dat_attn$longstring_run) & dat_attn$longstring_run > thr

dat_attn$exclude_reason <- ifelse(
  is.na(dat_attn$exclude_reason) & flag, "longstring_outlier",
  ifelse(!is.na(dat_attn$exclude_reason) & flag,
         paste(dat_attn$exclude_reason, "+ longstring_outlier"),
         dat_attn$exclude_reason)
)

dat_final <- dat_attn[!flag, , drop = FALSE]

cat("Longstring on First..Seventh: threshold >", round(thr, 2),
    "| removed", sum(flag, na.rm = TRUE), "rows.\n")
```

```
## Longstring on First..Seventh: threshold > 4.5 | removed 34 rows.
```

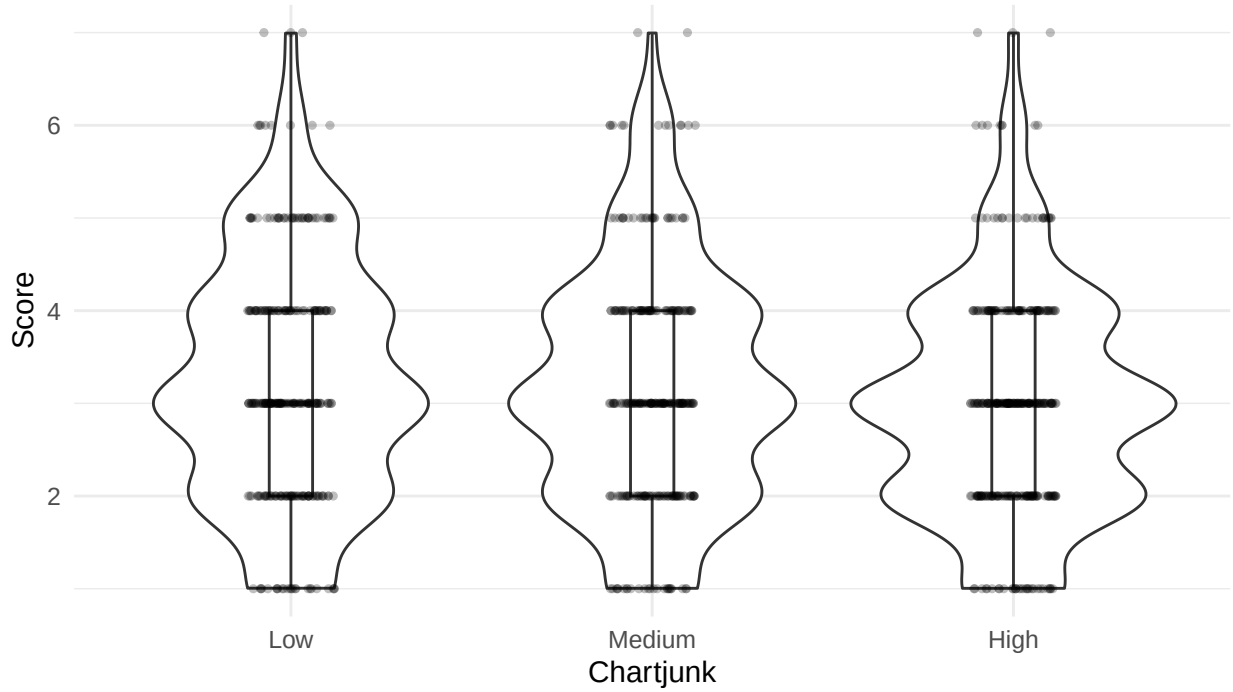
8) Quick EDA (optional)

```
# Score ~ hierarchy
ggplot(dat_final, aes(hierarchy, score)) +
  geom_violin(trim = FALSE) +
  geom_boxplot(width = .12, outlier.shape = NA) +
  geom_jitter(width = .12, height = 0, alpha = .25, size = 1) +
  labs(y = "Score", x = "Hierarchy") +
  theme_minimal(base_size = 12)
```




```
# Score ~ chartjunk
ggplot(dat_final, aes(chartjunk, score)) +
  geom_violin(trim = FALSE) +
  geom_boxplot(width = .12, outlier.shape = NA) +
  geom_jitter(width = .12, height = 0, alpha = .25, size = 1) +
  labs(x = "Chartjunk", y = "Score") +
  ylim (c(1,7))+
  theme_minimal(base_size = 12)
```

```
## Warning: Removed 420 rows containing missing values or values outside the scale range
## (`geom_violin()`).
```



9) Sanity tables

```
table(dat_attn$exclude_reason, useNA = "ifany")
```

```
##
## longstring_outlier               <NA>
##                 34                849
```

```
summary(dat_attn$longstring_run)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   2.000   2.000   2.345   3.000   7.000
```

```
# Peek at the First..Seventh encodings
head(dat_attn[, c("FirstItem","SecondItem","ThirdItem","FourthItem","FifthItem","SixthItem","SeventhIte
```

```
## # A tibble: 10 x 7
##    FirstItem SecondItem ThirdItem FourthItem FifthItem SixthItem SeventhItem
##        <int>      <int>     <int>      <int>     <int>     <int>        <int>
##  1         3          2         3          1         1         2            2
##  2         3          4         1          2         1         1            4
##  3         1          4         1          4         1         2            4
##  4         1          4         1          3         3         3            2
##  5         2          2         3          3         4         3            1
##  6         3          2         2          4         2         1            2
##  7         3          4         1          1         1         3            2
```

```
##  8        3         2        1         1        1        3         4
##  9        2         3        3         2        1        3         2
## 10        3         1        2         2        2        3         1
```

10) Session info

```
sessionInfo()
```

```
## R version 4.4.2 (2024-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26100)
##
## Matrix products: default
##
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/Los_Angeles
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] careless_1.2.2   effectsize_1.0.0 broom_1.0.7      ggplot2_4.0.0
## [5] stringr_1.5.1    dplyr_1.1.4      readr_2.1.5
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.3     tidyr_1.3.1        stringi_1.8.4      lattice_0.22-6
##  [5] hms_1.1.3          digest_0.6.37      magrittr_2.0.3     evaluate_1.0.3
##  [9] grid_4.4.2         estimability_1.5.1 RColorBrewer_1.1-3 mvtnorm_1.3-3
## [13] fastmap_1.2.0      backports_1.5.0    purrr_1.0.2        scales_1.4.0
## [17] mnormt_2.1.1       cli_3.6.3          crayon_1.5.3       rlang_1.1.5
## [21] bit64_4.6.0-1      withr_3.0.2        yaml_2.3.10        tools_4.4.2
## [25] datawizard_1.0.0   parallel_4.4.2     tzdb_0.4.0         pacman_0.5.1
## [29] bayestestR_0.15.1  vctrs_0.6.5        R6_2.5.1           lifecycle_1.0.4
## [33] emmeans_1.11.2-8   bit_4.5.0.1        vroom_1.6.5        psych_2.5.3
## [37] insight_1.0.1      pkgconfig_2.0.3    pillar_1.10.1      gtable_0.3.6
## [41] glue_1.8.0         xfun_0.50          tibble_3.2.1       tidyselect_1.2.1
## [45] rstudioapi_0.17.1  parameters_0.24.1  knitr_1.49         farver_2.1.2
## [49] xtable_1.8-4       nlme_3.1-166       htmltools_0.5.8.1  labeling_0.4.3
## [53] rmarkdown_2.29     compiler_4.4.2     S7_0.2.0
```

```
# ---- ANCOVA: adjust for dashboard use (Never vs Otherwise) ----
suppressPackageStartupMessages({
  library(car)
```

```
  library(emmeans)
  library(effectsize)
  library(ggplot2)
})
```

```
## Warning: package 'car' was built under R version 4.4.3
```

```
## Warning: package 'carData' was built under R version 4.4.3
```

```
## Warning: package 'emmeans' was built under R version 4.4.3
```

```
# 1) Recode usage to binary factor (ref = Otherwise)
dat_final$use_never <- ifelse(tolower(trimws(dat_final$DataDashboardUsage)) == "never",
                              "Never", "Otherwise")
dat_final$use_never <- factor(dat_final$use_never, levels = c("Otherwise","Never"))

# (Optional) sanity check
table(dat_final$use_never, useNA = "ifany")
```

```
##
## Otherwise     Never
##       243       606
```

```
# 2) Fit ANCOVA (binary covariate)
m_anc <- lm(score ~ hierarchy * chartjunk + use_never, data = dat_final)

# 3) Type-III tests with sum contrasts
old_contr <- options(contrasts = c("contr.sum","contr.poly"))
anc_tests <- car::Anova(m_anc, type = 3)
options(old_contr)

anc_tests
```

```
## Anova Table (Type III tests)
##
## Response: score
##                      Sum Sq  Df  F value    Pr(>F)
## (Intercept)         1490.94   1 920.3100 < 2.2e-16 ***
## hierarchy             41.53   3   8.5443  1.36e-05 ***
## chartjunk              5.60   2   1.7297    0.1780
## use_never              1.06   1   0.6523    0.4195
## hierarchy:chartjunk    3.46   6   0.3557    0.9067
## Residuals           1354.36 836
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# 4) Effect sizes (partial eta^2)
eta_squared(anc_tests, partial = TRUE)
```

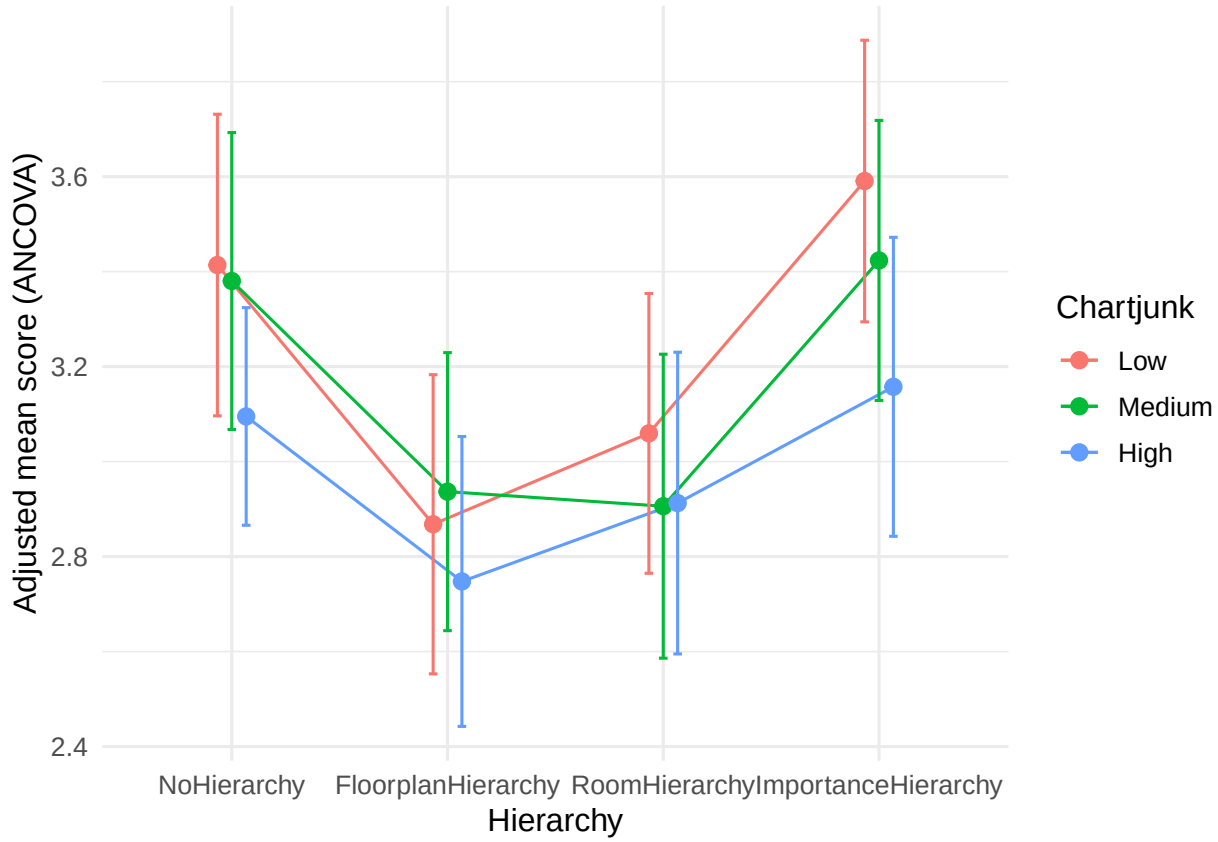
```
## Type 3 ANOVAs only give sensible and informative results when covariates
##   are mean-centered and factors are coded with orthogonal contrasts (such
##   as those produced by `contr.sum`, `contr.poly`, or `contr.helmert`, but
##   *not* by the default `contr.treatment`).
```

```
## # Effect Size for ANOVA (Type III)
##
## Parameter           | Eta2 (partial) |       95% CI
## -------------------------------------------------
## hierarchy           |           0.03 | [0.01, 1.00]
## chartjunk           |       4.12e-03 | [0.00, 1.00]
## use_never           |       7.80e-04 | [0.00, 1.00]
## hierarchy:chartjunk |       2.55e-03 | [0.00, 1.00]
##
## - One-sided CIs: upper bound fixed at [1.00].
```

```r
# 5) Adjusted means (EMMs) for hierarchy x chartjunk
#    weights = "proportional" averages over use_never by its sample proportions.
#    If you prefer equal weighting across use_never levels, set weights = "equal".
emm_anc <- emmeans(m_anc, ~ hierarchy * chartjunk, weights = "proportional")
confint(emm_anc)
```

```
##  hierarchy           chartjunk emmean    SE  df lower.CL upper.CL
##  NoHierarchy         Low         3.41 0.162 836     3.10     3.73
##  FloorplanHierarchy  Low         2.87 0.160 836     2.55     3.18
##  RoomHierarchy       Low         3.06 0.150 836     2.76     3.35
##  ImportanceHierarchy Low         3.59 0.151 836     3.29     3.89
##  NoHierarchy         Medium      3.38 0.159 836     3.07     3.69
##  FloorplanHierarchy  Medium      2.94 0.149 836     2.64     3.23
##  RoomHierarchy       Medium      2.91 0.163 836     2.59     3.23
##  ImportanceHierarchy Medium      3.42 0.150 836     3.13     3.72
##  NoHierarchy         High        3.10 0.117 836     2.87     3.32
##  FloorplanHierarchy  High        2.75 0.156 836     2.44     3.05
##  RoomHierarchy       High        2.91 0.162 836     2.59     3.23
##  ImportanceHierarchy High        3.16 0.160 836     2.84     3.47
##
## Results are averaged over the levels of: use_never
## Confidence level used: 0.95
```

```r
# 6) Plot adjusted means with 95% CIs
emm_df <- as.data.frame(confint(emm_anc))
ggplot(emm_df, aes(hierarchy, emmean, group = chartjunk, color = chartjunk)) +
  geom_line(position = position_dodge(width = .2)) +
  geom_point(position = position_dodge(width = .2), size = 2.5) +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL),
                width = .12, position = position_dodge(width = .2)) +
  labs(x = "Hierarchy", y = "Adjusted mean score (ANCOVA)",
       color = "Chartjunk") +
  theme_minimal(base_size = 12)
```




```
# 7) (Optional) follow-ups
# Pairwise simple effects of chartjunk within each hierarchy (Tukey-adjusted)
pairs(emmeans(m_anc, ~ chartjunk | hierarchy, weights = "proportional"),
      adjust = "tukey")
```

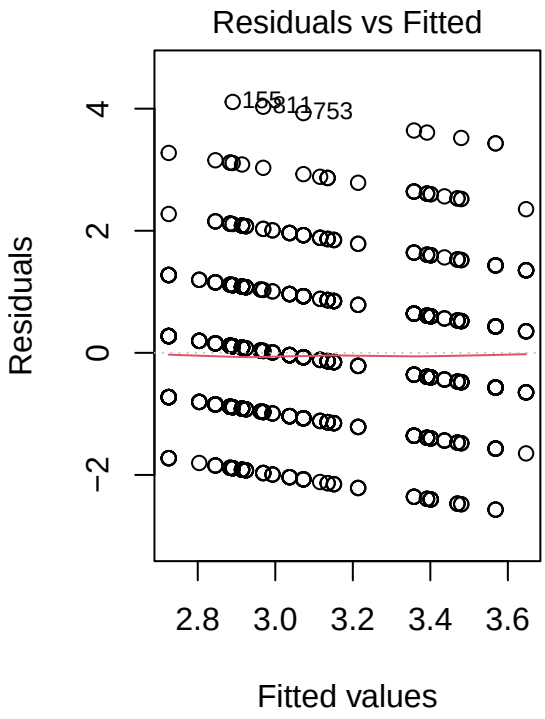
```
## hierarchy = NoHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium   0.03365 0.227 836   0.148  0.9880
##  Low - High     0.31883 0.200 836   1.597  0.2473
##  Medium - High  0.28519 0.197 836   1.445  0.3182
##
## hierarchy = FloorplanHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium  -0.06872 0.219 836  -0.313  0.9473
##  Low - High     0.12041 0.224 836   0.539  0.8522
##  Medium - High  0.18913 0.215 836   0.878  0.6543
##
## hierarchy = RoomHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium   0.15340 0.221 836   0.693  0.7679
##  Low - High     0.14681 0.221 836   0.665  0.7841
##  Medium - High -0.00659 0.230 836  -0.029  0.9995
##
## hierarchy = ImportanceHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
```

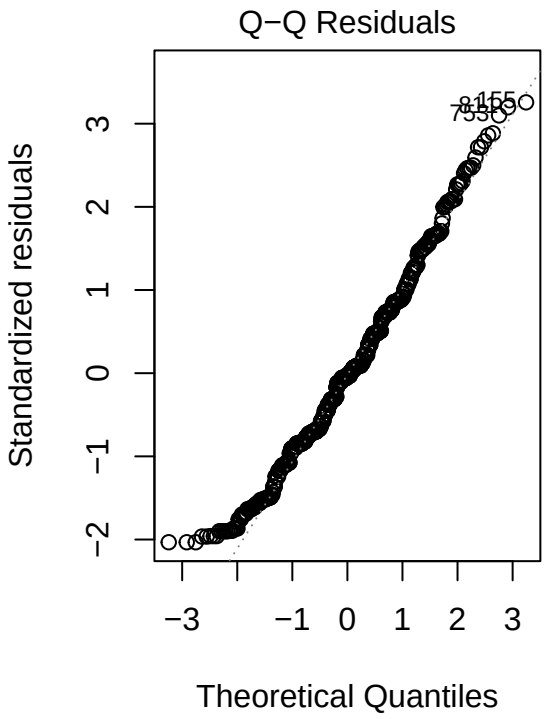
```
##  Low - Medium   0.16723 0.213 836   0.785  0.7123
##  Low - High     0.43328 0.220 836   1.967  0.1212
##  Medium - High  0.26605 0.220 836   1.211  0.4470
##
## Results are averaged over the levels of: use_never
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```
# Pairwise simple effects of hierarchy within each chartjunk
pairs(emmeans(m_anc, ~ hierarchy | chartjunk, weights = "proportional"),
      adjust = "tukey")
```

```
## chartjunk = Low:
##  contrast                                estimate    SE  df t.ratio p.value
##  NoHierarchy - FloorplanHierarchy          0.5459 0.228 836   2.397  0.0783
##  NoHierarchy - RoomHierarchy               0.3545 0.221 836   1.605  0.3762
##  NoHierarchy - ImportanceHierarchy        -0.1768 0.221 836  -0.799  0.8549
##  FloorplanHierarchy - RoomHierarchy       -0.1914 0.220 836  -0.871  0.8200
##  FloorplanHierarchy - ImportanceHierarchy -0.7227 0.220 836  -3.280  0.0059
##  RoomHierarchy - ImportanceHierarchy      -0.5313 0.213 836  -2.495  0.0614
##
## chartjunk = Medium:
##  contrast                                estimate    SE  df t.ratio p.value
##  NoHierarchy - FloorplanHierarchy          0.4435 0.218 836   2.035  0.1760
##  NoHierarchy - RoomHierarchy               0.4742 0.228 836   2.082  0.1598
##  NoHierarchy - ImportanceHierarchy        -0.0433 0.219 836  -0.197  0.9973
##  FloorplanHierarchy - RoomHierarchy        0.0307 0.221 836   0.139  0.9990
##  FloorplanHierarchy - ImportanceHierarchy -0.4868 0.212 836  -2.297  0.0996
##  RoomHierarchy - ImportanceHierarchy      -0.5175 0.222 836  -2.332  0.0918
##
## chartjunk = High:
##  contrast                                estimate    SE  df t.ratio p.value
##  NoHierarchy - FloorplanHierarchy          0.3475 0.194 836   1.787  0.2801
##  NoHierarchy - RoomHierarchy               0.1824 0.200 836   0.914  0.7976
##  NoHierarchy - ImportanceHierarchy        -0.0624 0.198 836  -0.315  0.9892
##  FloorplanHierarchy - RoomHierarchy       -0.1650 0.225 836  -0.735  0.8830
##  FloorplanHierarchy - ImportanceHierarchy -0.4099 0.223 836  -1.835  0.2577
##  RoomHierarchy - ImportanceHierarchy      -0.2448 0.228 836  -1.075  0.7050
##
## Results are averaged over the levels of: use_never
## P value adjustment: tukey method for comparing a family of 4 estimates
```

```
# 8) (Optional) model diagnostics
par(mfrow = c(1,2))
plot(m_anc, which = 1)  # Residuals vs Fitted
plot(m_anc, which = 2)  # Normal Q-Q
```

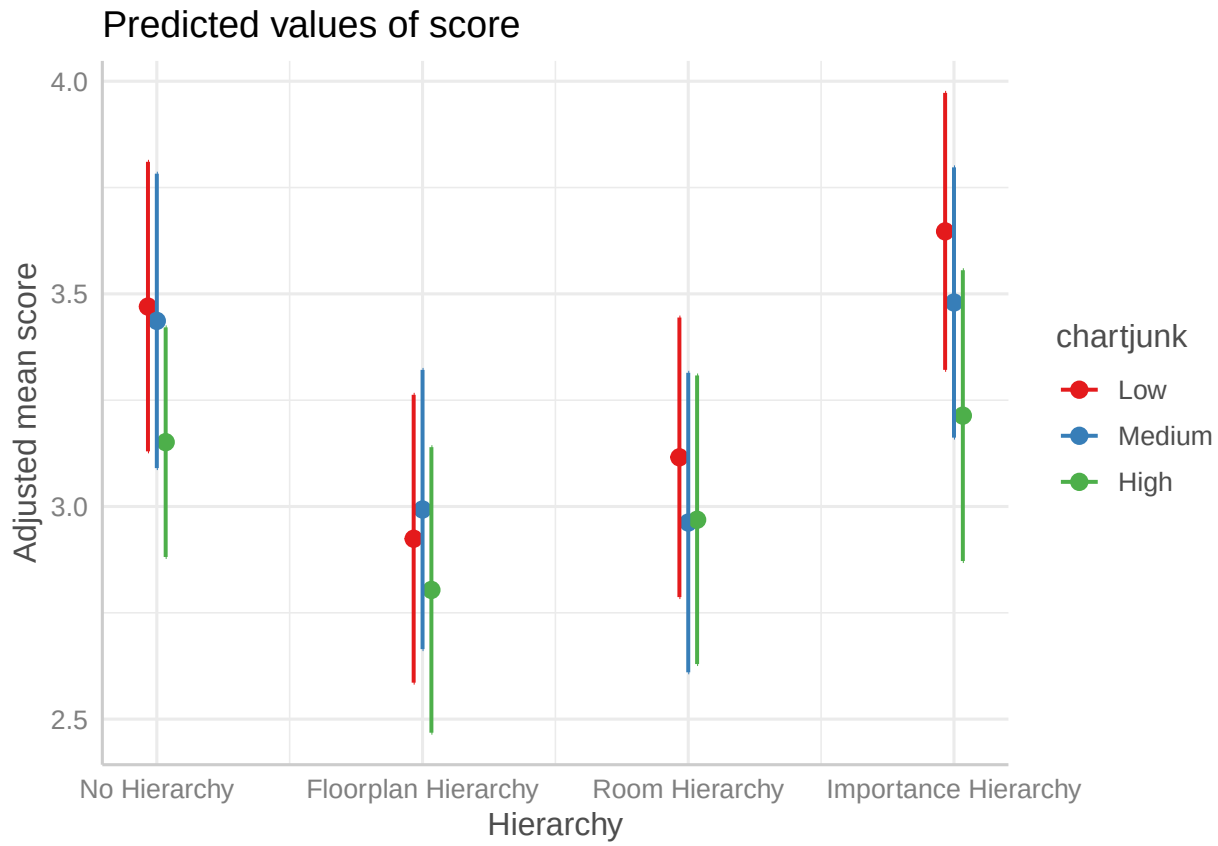





```
par(mfrow = c(1,1))
```

```
# ---- sjPlot visuals (ANCOVA) ----
pacman::p_load(sjPlot, sjlabelled)

# Optional: consistent look + quiet color codes in console
theme_set(sjPlot::theme_sjplot())
options(cli.num_colors = 1, crayon.enabled = FALSE)

# 1) Interaction: hierarchy × chartjunk (averages over use_never by sample mix)
p_int <- sjPlot::plot_model(
  m_anc,
  type  = "int",
  terms = c("hierarchy", "chartjunk"),   # x, group
  ci.lvl = 0.95
) + labs(x = "Hierarchy", y = "Adjusted mean score") +
    guides(linetype = "none", shape = "none")
p_int
```

```
## Ignoring unknown labels:
## * linetype : "chartjunk"
## * shape : "chartjunk"
```



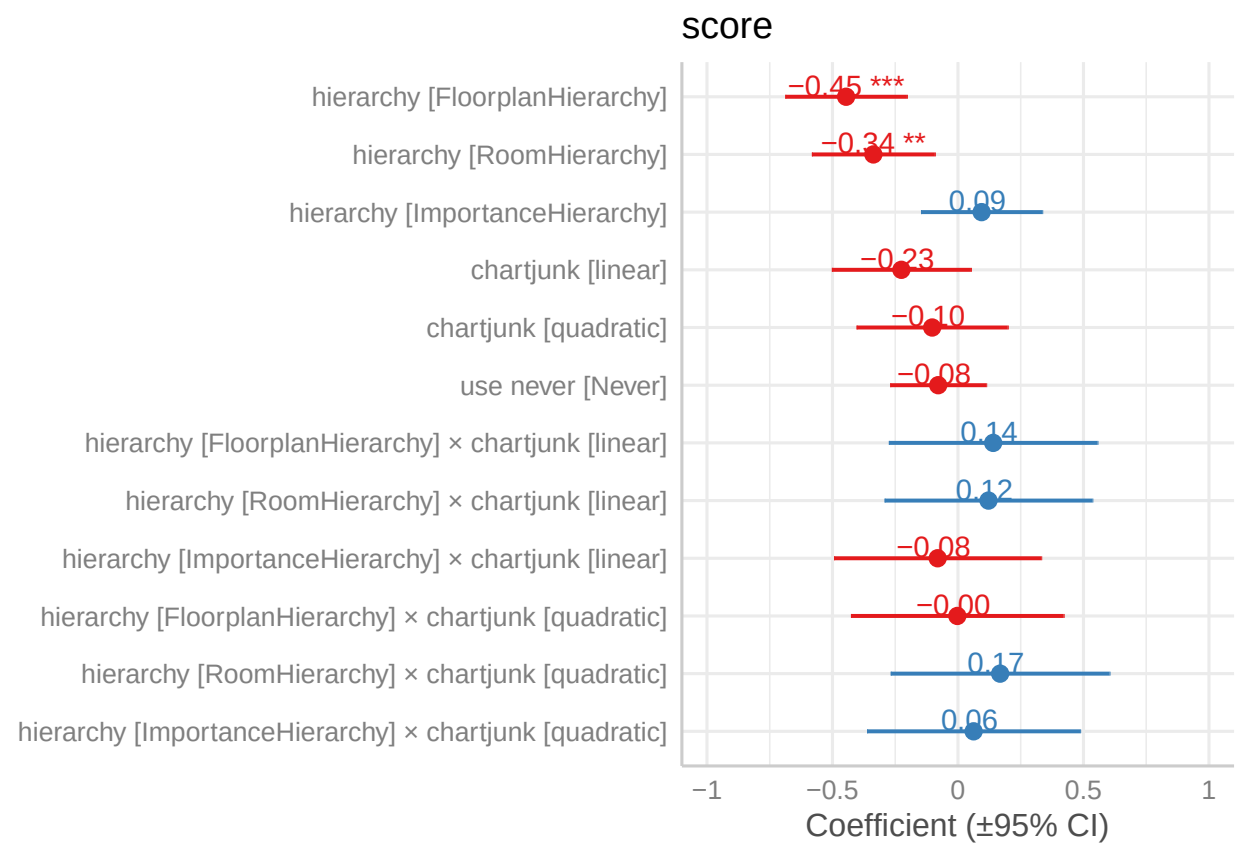

```
# 2) Predicted means, explicitly showing both use_never levels
# terms = c(x, group, facet)
p_pred <- sjPlot::plot_model(
  m_anc,
  type  = "pred",
  terms = c("hierarchy", "chartjunk", "use_never")
) + labs(x = "Hierarchy", y = "Adjusted mean score", color = "Chartjunk") +
    guides(linetype = "none", shape = "none")
p_pred
```

```
## Ignoring unknown labels:
## * linetype : "chartjunk"
## * shape : "chartjunk"
```


```
# 3) Coefficient forest (fixed effects with 95% CI)
p_coef <- sjPlot::plot_model(
  m_anc,
  type = "est",          # estimated coefficients
  show.values = TRUE,
  value.offset = .2
) + labs(x = NULL, y = "Coefficient (±95% CI)")
```

```
## Warning: Using ‘size‘ aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use ‘linewidth‘ instead.
## i The deprecated feature was likely used in the sjPlot package.
##   Please report the issue at <https://github.com/strengejacke/sjPlot/issues>.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.

## Warning: ‘aes_string()‘ was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with ‘aes()‘.
## i See also ‘vignette("ggplot2-in-packages")‘ for more information.
## i The deprecated feature was likely used in the sjPlot package.
##   Please report the issue at <https://github.com/strengejacke/sjPlot/issues>.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```

```
p_coef
```




```
# 4) Model table (nice HTML output in the notebook)
sjPlot::tab_model(
  m_anc,
  dv.labels = "Score (ANCOVA)",
  show.ci = TRUE,
  show.std = TRUE,
  p.style = "stars",
  string.pred = "Predictor",
  string.est  = "b"
)
```

Score (ANCOVA)

Predictor

b

std. Beta

CI

standardized CI

(Intercept)

3.35 ***

0.18

-Inf – Inf

-Inf – Inf

hierarchy[FloorplanHierarchy]

-0.45 ***

-0.35

-Inf – Inf

-Inf – Inf

hierarchy [RoomHierarchy]

-0.34 **

-0.26

-Inf – Inf

-Inf – Inf

hierarchy[ImportanceHierarchy]

0.09

0.07

-Inf – Inf

-Inf – Inf

chartjunk [linear]

-0.23

-0.17

-Inf – Inf

-Inf – Inf

chartjunk [quadratic]

-0.10

-0.08

-Inf – Inf

-Inf – Inf

use never [Never]

-0.08

-0.06

-Inf – Inf

-Inf – Inf

hierarchy[FloorplanHierarchy] ×chartjunk [linear]

0.14

0.11

-Inf – Inf

-Inf – Inf

hierarchy [RoomHierarchy]× chartjunk [linear]

0.12

0.09

-Inf – Inf

-Inf – Inf

hierarchy[ImportanceHierarchy] ×chartjunk [linear]

-0.08

-0.06

-Inf – Inf

-Inf – Inf

hierarchy[FloorplanHierarchy] ×chartjunk [quadratic]

-0.00

-0.00

-Inf – Inf

-Inf – Inf

hierarchy [RoomHierarchy]× chartjunk [quadratic]

0.17

0.13

-Inf – Inf

-Inf – Inf

hierarchy[ImportanceHierarchy] ×chartjunk [quadratic]

0.06

0.05

-Inf – Inf

-Inf – Inf

Observations

849

R2 / R2 adjusted

0.039 / 0.025

- p<0.05 ** p<0.01 *** p<0.001

```
m_mod <- lm(score ~ hierarchy * chartjunk * use_never, data = dat_final)
car::Anova(m_mod, type = 3)
```

```
## Anova Table (Type III tests)
##
## Response: score
##                               Sum Sq  Df  F value  Pr(>F)
## (Intercept)                   651.77   1 400.3925 < 2e-16 ***
## hierarchy                      15.98   3   3.2728 0.02066 *
## chartjunk                       0.55   2   0.1705 0.84330
## use_never                       0.00   1   0.0006 0.98125
## hierarchy:chartjunk             4.98   6   0.5095 0.80144
## hierarchy:use_never             1.07   3   0.2194 0.88296
## chartjunk:use_never             0.55   2   0.1677 0.84561
## hierarchy:chartjunk:use_never   5.66   6   0.5793 0.74703
## Residuals                    1342.97 825
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
m_aov <- lm(score ~ hierarchy * chartjunk, data = dat_final)
m_anc <- lm(score ~ hierarchy * chartjunk + use_never, data = dat_final)

anova(m_aov, m_anc)   # nested F-test
```

```
## Analysis of Variance Table
##
## Model 1: score ~ hierarchy * chartjunk
## Model 2: score ~ hierarchy * chartjunk + use_never
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1    837 1355.4
## 2    836 1354.4  1    1.0568 0.6523 0.4195
```

```
AIC(m_aov, m_anc); BIC(m_aov, m_anc)
```

```
##       df      AIC
## m_aov 13 2832.522
## m_anc 14 2833.860
```

```
##       df      BIC
## m_aov 13 2894.195
## m_anc 14 2900.276
```

```
# --- Final 2×3 ANOVA (no covariate) ---
library(car); library(emmeans); library(effectsize); library(sjPlot)

m_aov <- lm(score ~ hierarchy * chartjunk, data = dat_final)

# Type III with sum contrasts (reporting-friendly)
oldc <- options(contrasts = c("contr.sum","contr.poly"))
aov_tests <- car::Anova(m_aov, type = 3)
options(oldc)

aov_tests
```

```
## Anova Table (Type III tests)
```

```
##
## Response: score
##                       Sum Sq  Df   F value    Pr(>F)
## (Intercept)         2434.14   1 1503.1388 < 2.2e-16 ***
## hierarchy             41.91   3    8.6278  1.21e-05 ***
## chartjunk              5.73   2    1.7699    0.1710
## hierarchy:chartjunk    3.50   6    0.3600    0.9042
## Residuals           1355.41 837
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
effectsize::eta_squared(aov_tests, partial = TRUE)
```

```
## Type 3 ANOVAs only give sensible and informative results when covariates
##   are mean-centered and factors are coded with orthogonal contrasts (such
##   as those produced by `contr.sum`, `contr.poly`, or `contr.helmert`, but
##   *not* by the default `contr.treatment`).

## # Effect Size for ANOVA (Type III)
##
## Parameter           | Eta2 (partial) |       95% CI
## -------------------------------------------------
## hierarchy           |           0.03 | [0.01, 1.00]
## chartjunk           |       4.21e-03 | [0.00, 1.00]
## hierarchy:chartjunk |       2.57e-03 | [0.00, 1.00]
##
## - One-sided CIs: upper bound fixed at [1.00].
```

```
# EMMs + pairwise (Tukey)
emm <- emmeans(m_aov, ~ hierarchy * chartjunk)
pairs(emmeans(m_aov, ~ chartjunk | hierarchy), adjust = "tukey")
```

```
## hierarchy = NoHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium    0.0444 0.227 837   0.196  0.9791
##  Low - High      0.3269 0.199 837   1.640  0.2294
##  Medium - High   0.2826 0.197 837   1.432  0.3247
##
## hierarchy = FloorplanHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium   -0.0585 0.219 837  -0.267  0.9614
##  Low - High      0.1267 0.223 837   0.568  0.8374
##  Medium - High   0.1852 0.215 837   0.860  0.6654
##
## hierarchy = RoomHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium    0.1539 0.221 837   0.695  0.7664
##  Low - High      0.1362 0.220 837   0.618  0.8104
##  Medium - High  -0.0177 0.229 837  -0.077  0.9967
##
## hierarchy = ImportanceHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
```

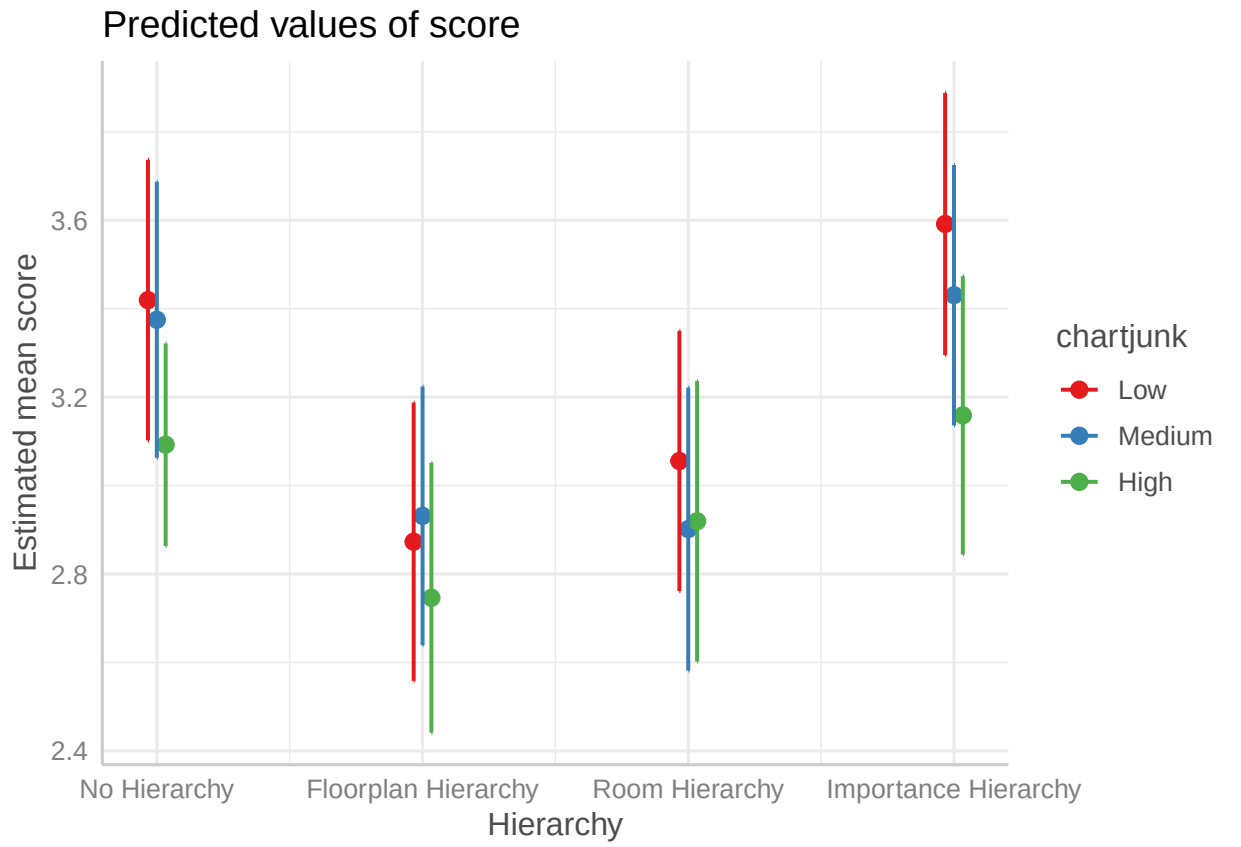
```
##  Low - Medium    0.1610 0.213 837   0.756  0.7298
##  Low - High      0.4328 0.220 837   1.965  0.1216
##  Medium - High   0.2718 0.220 837   1.238  0.4310
##
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```
pairs(emmeans(m_aov, ~ hierarchy | chartjunk), adjust = "tukey")
```

```
## chartjunk = Low:
##  contrast                                estimate    SE  df t.ratio p.value
##  NoHierarchy - FloorplanHierarchy          0.5463 0.228 837   2.400  0.0779
##  NoHierarchy - RoomHierarchy               0.3638 0.220 837   1.650  0.3511
##  NoHierarchy - ImportanceHierarchy        -0.1722 0.221 837  -0.778  0.8642
##  FloorplanHierarchy - RoomHierarchy       -0.1825 0.220 837  -0.831  0.8395
##  FloorplanHierarchy - ImportanceHierarchy -0.7185 0.220 837  -3.262  0.0063
##  RoomHierarchy - ImportanceHierarchy      -0.5360 0.213 837  -2.518  0.0578
##
## chartjunk = Medium:
##  contrast                                estimate    SE  df t.ratio p.value
##  NoHierarchy - FloorplanHierarchy          0.4435 0.218 837   2.035  0.1759
##  NoHierarchy - RoomHierarchy               0.4734 0.228 837   2.079  0.1609
##  NoHierarchy - ImportanceHierarchy        -0.0556 0.219 837  -0.254  0.9942
##  FloorplanHierarchy - RoomHierarchy        0.0299 0.221 837   0.135  0.9991
##  FloorplanHierarchy - ImportanceHierarchy -0.4990 0.211 837  -2.361  0.0855
##  RoomHierarchy - ImportanceHierarchy      -0.5289 0.221 837  -2.388  0.0801
##
## chartjunk = High:
##  contrast                                estimate    SE  df t.ratio p.value
##  NoHierarchy - FloorplanHierarchy          0.3462 0.194 837   1.781  0.2832
##  NoHierarchy - RoomHierarchy               0.1731 0.199 837   0.868  0.8212
##  NoHierarchy - ImportanceHierarchy        -0.0663 0.198 837  -0.334  0.9871
##  FloorplanHierarchy - RoomHierarchy       -0.1731 0.224 837  -0.772  0.8672
##  FloorplanHierarchy - ImportanceHierarchy -0.4125 0.223 837  -1.847  0.2522
##  RoomHierarchy - ImportanceHierarchy      -0.2394 0.228 837  -1.052  0.7191
##
## P value adjustment: tukey method for comparing a family of 4 estimates
```

```
# Plot (report-friendly)
theme_set(sjPlot::theme_sjplot())
sjPlot::plot_model(m_aov, type = "int",
                   terms = c("hierarchy","chartjunk"), ci.lvl = 0.95) +
  labs(x = "Hierarchy", y = "Estimated mean score") +
  guides(linetype = "none", shape = "none")
```

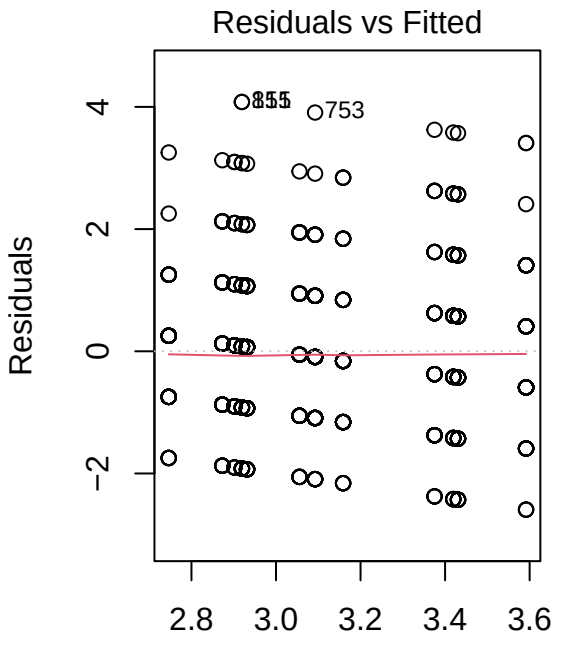
```
## Ignoring unknown labels:
## * linetype : "chartjunk"
## * shape : "chartjunk"
```

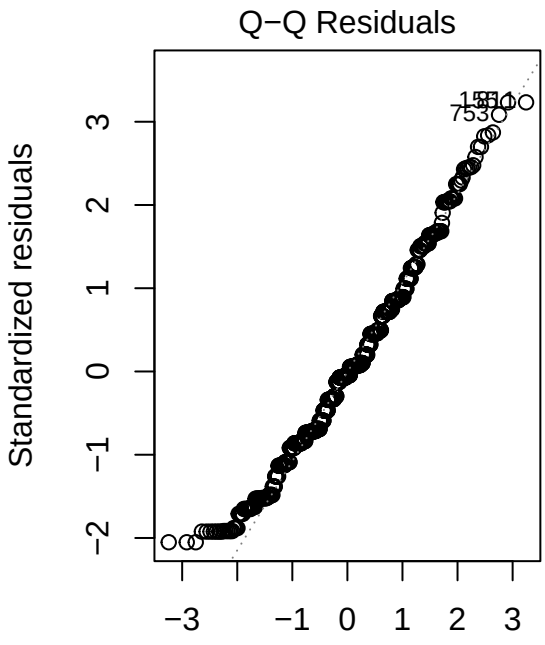



```
# (Optional) basic assumptions
car::leveneTest(score ~ interaction(hierarchy, chartjunk), data = dat_final)
```

```
## Levene's Test for Homogeneity of Variance (center = median)
##        Df F value  Pr(>F)
## group  11  1.7062 0.06746 .
##       837
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
par(mfrow = c(1,2)); plot(m_aov, which = 1); plot(m_aov, which = 2); par(mfrow = c(1,1))
```






```r
# 1) Omnibus 2×3 ANOVA + effect sizes
options(contrasts = c("contr.sum","contr.poly"))
m_aov <- lm(score ~ hierarchy * chartjunk, data = dat_final)
aov_tab <- car::Anova(m_aov, type = 3); aov_tab
```

```
## Anova Table (Type III tests)
##
## Response: score
##                      Sum Sq  Df   F value    Pr(>F)
## (Intercept)         8037.6   1 4963.4050 < 2.2e-16 ***
## hierarchy             41.9   3    8.6278  1.21e-05 ***
## chartjunk              9.7   2    2.9873   0.05096 .
## hierarchy:chartjunk    3.5   6    0.3600   0.90418
## Residuals           1355.4 837
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
effectsize::eta_squared(aov_tab, partial = TRUE)
```

```
## Type 3 ANOVAs only give sensible and informative results when covariates
##   are mean-centered and factors are coded with orthogonal contrasts (such
##   as those produced by `contr.sum`, `contr.poly`, or `contr.helmert`, but
##   *not* by the default `contr.treatment`).

## # Effect Size for ANOVA (Type III)
```

```
##
## Parameter           | Eta2 (partial) |       95% CI
## -------------------------------------------------
## hierarchy           |           0.03 | [0.01, 1.00]
## chartjunk           |       7.09e-03 | [0.00, 1.00]
## hierarchy:chartjunk |       2.57e-03 | [0.00, 1.00]
##
## - One-sided CIs: upper bound fixed at [1.00].
```

```
# 2) Planned linear trend for chartjunk (it's ordered)
# overall (balanced over hierarchy) and within each hierarchy
library(emmeans)
emm_cj   <- emmeans(m_aov, ~ chartjunk, weights = "equal")
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
contrast(emm_cj, "poly")              # look at the Linear row
```

```
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.256 0.108 837  -2.367  0.0182
##  quadratic   -0.105 0.189 837  -0.556  0.5781
##
## Results are averaged over the levels of: hierarchy
```

```
emm_cj_h <- emmeans(m_aov, ~ chartjunk | hierarchy, weights = "equal")
contrast(emm_cj_h, "poly")            # Linear per-hierarchy
```

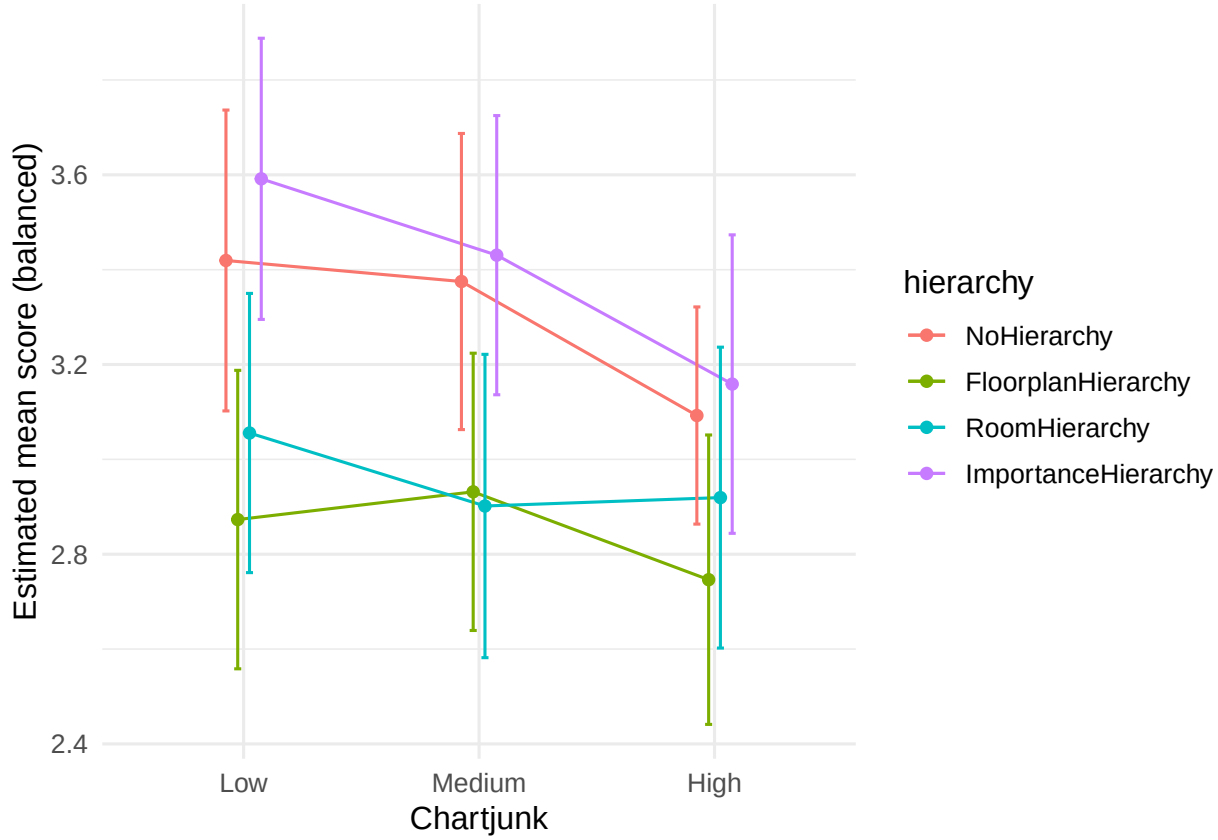
```
## hierarchy = NoHierarchy:
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.327 0.199 837  -1.640  0.1013
##  quadratic   -0.238 0.375 837  -0.635  0.5259
##
## hierarchy = FloorplanHierarchy:
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.127 0.223 837  -0.568  0.5705
##  quadratic   -0.244 0.372 837  -0.655  0.5129
##
## hierarchy = RoomHierarchy:
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.136 0.220 837  -0.618  0.5369
##  quadratic    0.172 0.393 837   0.436  0.6628
##
## hierarchy = ImportanceHierarchy:
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.433 0.220 837  -1.965  0.0497
##  quadratic   -0.111 0.372 837  -0.298  0.7659
```

```
# 3) Simple effects of chartjunk within each hierarchy (Tukey)
pairs(emm_cj_h, adjust = "tukey")
```

```
## hierarchy = NoHierarchy:
```

```
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium    0.0444 0.227 837   0.196  0.9791
##  Low - High      0.3269 0.199 837   1.640  0.2294
##  Medium - High   0.2826 0.197 837   1.432  0.3247
##
## hierarchy = FloorplanHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium   -0.0585 0.219 837  -0.267  0.9614
##  Low - High      0.1267 0.223 837   0.568  0.8374
##  Medium - High   0.1852 0.215 837   0.860  0.6654
##
## hierarchy = RoomHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium    0.1539 0.221 837   0.695  0.7664
##  Low - High      0.1362 0.220 837   0.618  0.8104
##  Medium - High  -0.0177 0.229 837  -0.077  0.9967
##
## hierarchy = ImportanceHierarchy:
##  contrast      estimate    SE  df t.ratio p.value
##  Low - Medium    0.1610 0.213 837   0.756  0.7298
##  Low - High      0.4328 0.220 837   1.965  0.1216
##  Medium - High   0.2718 0.220 837   1.238  0.4310
##
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```r
# (Optional) visualize the monotone look you're seeing
emm_df <- as.data.frame(confint(emm_cj_h))
ggplot(emm_df, aes(chartjunk, emmean, group = hierarchy, color = hierarchy)) +
  geom_line(position = position_dodge(.2)) +
  geom_point(position = position_dodge(.2)) +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL),
                width = .12, position = position_dodge(.2)) +
  labs(x = "Chartjunk", y = "Estimated mean score (balanced)") +
  theme_minimal(base_size = 12)
```




```r
# ensure ordered levels
dat_final$chartjunk <- factor(dat_final$chartjunk,
                              levels = c("Low","Medium","High"),
                              ordered = TRUE)

# contrasts: sum for hierarchy (Type III friendly), polynomial for chartjunk
contrasts(dat_final$hierarchy) <- contr.sum(4)
contrasts(dat_final$chartjunk)  <- contr.poly(3)

# refit the 2×3 model
m_aov <- lm(score ~ hierarchy * chartjunk, data = dat_final)

# overall Type III tests
car::Anova(m_aov, type = 3)
```

```
## Anova Table (Type III tests)
##
## Response: score
##                     Sum Sq  Df   F value    Pr(>F)
## (Intercept)         8037.6   1 4963.4050 < 2.2e-16 ***
## hierarchy             41.9   3    8.6278  1.21e-05 ***
## chartjunk              9.7   2    2.9873   0.05096 .
## hierarchy:chartjunk    3.5   6    0.3600   0.90418
## Residuals           1355.4 837
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
library(emmeans)

# overall linear trend for chartjunk (balanced over hierarchy)
emm_cj <- emmeans(m_aov, ~ chartjunk, weights = "equal")
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```r
contrast(emm_cj, "poly")   # check the "Linear" row
```

```
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.256 0.108 837  -2.367  0.0182
##  quadratic   -0.105 0.189 837  -0.556  0.5781
##
## Results are averaged over the levels of: hierarchy
```

```r
# linear trend within each hierarchy
emm_cj_h <- emmeans(m_aov, ~ chartjunk | hierarchy, weights = "equal")
contrast(emm_cj_h, "poly") # Linear per hierarchy level
```

```
## hierarchy = NoHierarchy:
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.327 0.199 837  -1.640  0.1013
##  quadratic   -0.238 0.375 837  -0.635  0.5259
##
## hierarchy = FloorplanHierarchy:
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.127 0.223 837  -0.568  0.5705
##  quadratic   -0.244 0.372 837  -0.655  0.5129
##
## hierarchy = RoomHierarchy:
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.136 0.220 837  -0.618  0.5369
##  quadratic    0.172 0.393 837   0.436  0.6628
##
## hierarchy = ImportanceHierarchy:
##  contrast  estimate    SE  df t.ratio p.value
##  linear      -0.433 0.220 837  -1.965  0.0497
##  quadratic   -0.111 0.372 837  -0.298  0.7659
```

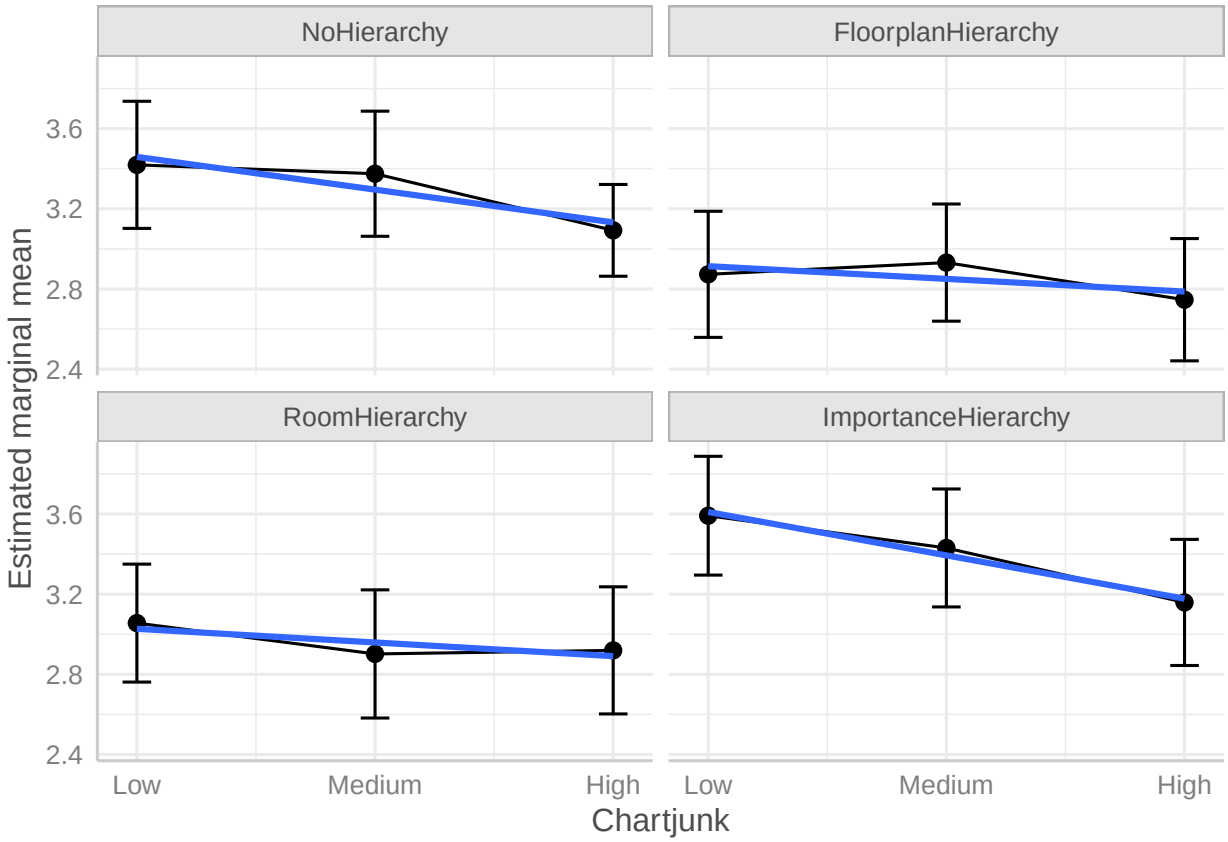
```r
# ---- linear contrast at a fixed hierarchy level -----------------------------
# deps
library(emmeans)
library(ggplot2)
library(sjPlot)  # for theme_sjplot()

plot_linear_contrast_at <- function(
  m,
  focal,                     # character: the factor you want the linear contrast on (e.g., "cond")
  moderator = "hierarchy",   # character: the factor to hold constant
  at_level = NULL,           # which level of moderator to hold fixed (e.g., "Manager")
  contrast = c("poly","custom"),  # "poly" for polynomial linear trend, or "custom"
```

```
  custom_weights = NULL,      # numeric vector if contrast = "custom" (length = #levels of focal)
  type = "response",          # EMM scale: "response" often nicer for GLMs
  emm_weights = "equal"       # how to average over other factors if present; "equal" is common
) {
  contrast <- match.arg(contrast)

  # discover default at_level if not provided
  mf <- tryCatch(model.frame(m), error = function(e) NULL)
  if (is.null(at_level)) {
    if (!is.null(mf) && moderator %in% names(mf) && is.factor(mf[[moderator]])) {
      at_level <- levels(mf[[moderator]])[1]
    } else {
      stop("Please provide 'at_level' (couldn't infer a factor level for ", moderator, ").")
    }
  }

  # build a reference grid with hierarchy held constant
  rg <- ref_grid(m, at = setNames(list(at_level), moderator), type = type)

  # EMMs for the focal factor at that hierarchy level
  emm <- emmeans(rg, as.formula(paste0("~ ", focal)), weights = emm_weights)

  # Linear contrast
  lin <- if (contrast == "poly") {
    # "poly" gives linear, quadratic, ...; take the linear component only
    contrast(emm, "poly")["linear"]
  } else {
    # custom weights you define in the order of levels(emm)
    if (is.null(custom_weights))
      stop("Provide 'custom_weights' when contrast = 'custom'.")
    if (length(custom_weights) != nrow(as.data.frame(emm)))
      stop("Length of 'custom_weights' must equal the number of levels in '", focal, "'.")
    contrast(emm, list(linear = custom_weights))
  }

  lin_sum <- summary(lin, infer = TRUE)
  tval <- lin_sum$t.ratio
  pval <- lin_sum$p.value

  # data frame for plotting
  emm_df <- as.data.frame(emm)
  emm_df$.xnum <- as.numeric(emm_df[[focal]])

  p <- ggplot(emm_df,
              aes(x = .xnum,
                  y = emmean,
                  ymin = lower.CL, ymax = upper.CL)) +
    geom_point(size = 3) +
    geom_line(group = 1) +
    geom_errorbar(width = 0.1) +
    geom_smooth(method = "lm", se = FALSE) +
    scale_x_continuous(breaks = emm_df$.xnum,
                       labels = emm_df[[focal]]) +
```

```
    labs(
      x = focal,
      y = paste("Estimated marginal mean (", type, " scale)", sep = ""),
      title = paste0("Linear contrast across ", focal,
                     " (", moderator, " = ", at_level, ")"),
      subtitle = paste0("t = ", round(tval, 2),
                        ", p = ", format.pval(pval, digits = 3, eps = 1e-4))
    ) +
    theme_sjplot()

  list(plot = p, emm = emm, linear_contrast = lin)
}
# ------------------------------------------------------------------------------

# EXAMPLE USAGE:
# pacman::p_load(lme4)
# m <- lmer(y ~ cond * hierarchy + (1|id), data = d)  # or lm()/glm()
# out <- plot_linear_contrast_at(m, focal = "cond", moderator = "hierarchy",
#                                at_level = "Manager", contrast = "poly",
#                                type = "response", emm_weights = "equal")
# out$plot
# # summary(out$linear_contrast)  # full stats table
```

```
# --- Linear contrasts for chartjunk while holding hierarchy constant ----

# pick which fitted model you want to use
# (no covariate 2×3)     or     (ANCOVA with use_never)
mod <- m_aov              # or: mod <- m_anc

# how emmeans averages over other factors not shown in the formula
# "equal" = simple average of cells; "proportional" = weighted by sample sizes
wts <- "equal"            # or: wts <- "proportional"

# Uses your existing m_aov (or m_anc), dat_final, and plot_linear_contrast_at()
emm_cj_h <- emmeans(mod, ~ chartjunk | hierarchy, weights = wts)
emm_df   <- as.data.frame(confint(emm_cj_h))
emm_df$xnum <- as.numeric(emm_df$chartjunk)

ggplot(emm_df, aes(x = xnum, y = emmean)) +
  facet_wrap(~ hierarchy) +
  geom_point(size = 2.5) +
  geom_line(group = 1) +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL), width = .12) +
  geom_smooth(method = "lm", se = FALSE) +
  scale_x_continuous(breaks = seq_along(levels(dat_final$chartjunk)),
                     labels = levels(dat_final$chartjunk)) +
  labs(x = "Chartjunk", y = "Estimated marginal mean") +
  theme_sjplot()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

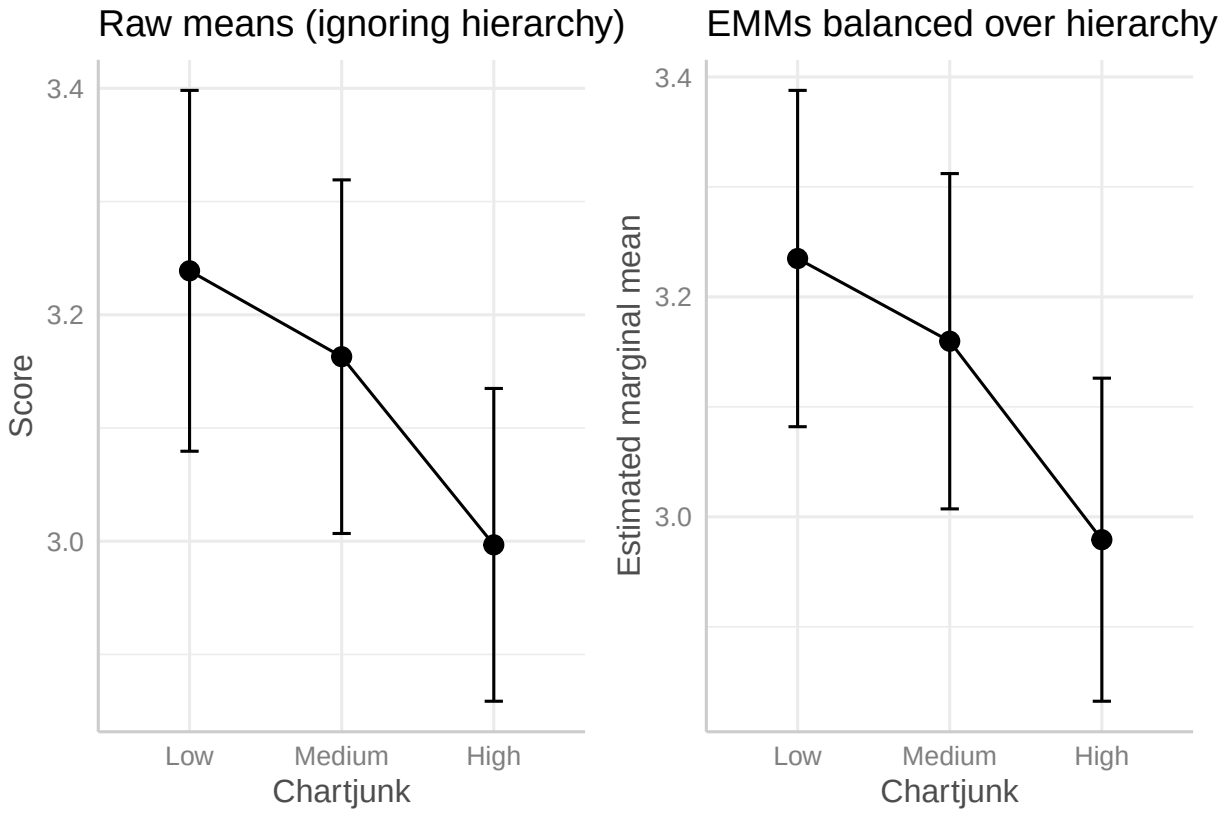



```
# --- 3-point line charts for chartjunk: raw vs. adjusted ----------------------
pacman::p_load(dplyr, ggplot2, emmeans, tibble, patchwork, sjPlot)

# Ensure ordered factor
dat_final$chartjunk <- factor(dat_final$chartjunk,
                              levels = c("Low","Medium","High"),
                              ordered = TRUE)

# If needed, fit (you already have m_aov from above)
if (!exists("m_aov")) {
  m_aov <- lm(score ~ hierarchy * chartjunk, data = dat_final)
}

# 1) RAW means (ignore hierarchy entirely)
raw_df <- dat_final %>%
  group_by(chartjunk) %>%
  summarize(
    n = sum(!is.na(score)),
    mean = mean(score, na.rm = TRUE),
    sd = sd(score, na.rm = TRUE),
    se = sd / sqrt(n),
    tcrit = qt(.975, df = pmax(n - 1, 1)),
    lower = mean - tcrit * se,
    upper = mean + tcrit * se,
    .groups = "drop"
  )
```

```
p_raw <- ggplot(raw_df, aes(x = chartjunk, y = mean, group = 1)) +
  geom_line() +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = .12) +
  labs(x = "Chartjunk", y = "Score", title = "Raw means (ignoring hierarchy)") +
  theme_sjplot()

# 2) ADJUSTED means (EMMs) for chartjunk, balanced over hierarchy
emm_cj_eq <- emmeans(m_aov, ~ chartjunk, weights = "equal")
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
emm_df <- as.data.frame(confint(emm_cj_eq))

p_adj <- ggplot(emm_df, aes(x = chartjunk, y = emmean, group = 1)) +
  geom_line() +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL), width = .12) +
  labs(x = "Chartjunk", y = "Estimated marginal mean",
       title = "EMMs balanced over hierarchy") +
  theme_sjplot()

# Show side by side
p_raw | p_adj
```



```r
# ------------------------------------------------------------------------------

# If you want the adjusted version to use sample-mix instead of equal weighting:
# emm_cj_prop <- emmeans(m_aov, ~ chartjunk, weights = "proportional")
# as.data.frame(confint(emm_cj_prop))
```